CÓDIGOCLARO · RUTA REACT

Tema 03 · Estado con useState

Guía de estudio · DAM/DAW

Estado local y actualizaciones funcionales.

1. Explicación

useState declara una pieza de estado para un componente y devuelve su valor actual y una función para solicitar un cambio. La llamada al setter programa un renderizado; no cambia la variable de la ejecución actual. Si el siguiente valor depende del anterior, usa la forma funcional setEstado(anterior => nuevoValor) para evitar cálculos con una captura obsoleta. Para objetos y arrays, crea una nueva referencia en vez de mutarlos.

2. Ejemplo

Código · jsx

```jsx
import { useState } from "react";

export default function Contador() {
  const [total, setTotal] = useState(0);
  return <button onClick={() => setTotal(valor => valor + 1)}>
    Pulsaciones: {total}
  </button>;
}
```

Resultado esperado

```
La interfaz empieza con Pulsaciones: 0 y sube una unidad por clic.
```

3. Errores frecuentes

- Cambiar directamente total o mutar el objeto que guarda el estado.
- Esperar que el valor local cambie inmediatamente después de llamar al setter.
- Usar total + 1 en varias actualizaciones acumuladas cuando dependen de la cola previa.

4. Ejercicio

Implementa un contador con botones para sumar y restar, sin permitir valores menores que cero.

Casos que debe superar

Entrada	Resultado esperado
inicializar	0
restar desde 0	0
sumar, sumar, restar	1

5. Solución explicada

Código · jsx

```jsx
const [valor, setValor] = useState(0);
const sumar = () => setValor(actual => actual + 1);
const restar = () => setValor(actual => Math.max(0, actual - 1));
```

Criterio de aceptación: Se aceptan reducer o dos setters si los eventos dan los mismos valores en secuencias repetidas.

Otras soluciones válidas

- Usar actual => actual + 1 para actualizaciones acumuladas.
- Usar useReducer si las transiciones de estado tienen varias reglas.

6. Fuentes oficiales

Equipo de React — State: A Component's Memory · Equipo de React — useState